

QUESTION 1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Aim

To implement dynamic memory allocation using malloc(), calloc(), realloc(), and free(), and detect memory leaks using Valgrind.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    /* malloc() */
    int *arr1 = (int *)malloc(5 * sizeof(int));

    if (arr1 == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        arr1[i] = i + 1;

    printf("malloc() allocated 5 integers\n");

    /* calloc() */
    int *arr2 = (int *)calloc(5, sizeof(int));

    if (arr2 == NULL) {
        printf("calloc failed\n");
        free(arr1);
        return 1;
    }

    printf("calloc() allocated 5 integers and initialized them to zero\n");
```

```

/* realloc() */
arr1 = (int *)realloc(arr1, 10 * sizeof(int));

if (arr1 == NULL) {
    printf("realloc failed\n");
    free(arr2);
    return 1;
}

for (i = 5; i < 10; i++)
    arr1[i] = i + 1;

printf("realloc() expanded arr1 to 10 integers\n");

/* Display values */
printf("arr1 values: ");
for (i = 0; i < 10; i++)
    printf("%d ", arr1[i]);

printf("\n");

/* Free arr1 */
free(arr1);

/* Free arr2 */
free(arr2);

/* Intentional memory leak */
int *leak = (int *)malloc(100 * sizeof(int));

if (leak != NULL) {
    leak[0] = 100;
    printf("Intentional memory leak created for Valgrind testing\n");
}

return 0;
}

```

Output

```

chakrika@blue:~$ cd ~/OSSP
chakrika@blue:~/OSSP$ mkdir -p Week8
chakrika@blue:~/OSSP$ cd Week8
chakrika@blue:~/OSSP/Week8$ nano memory_allocation.c
chakrika@blue:~/OSSP/Week8$ gcc -g memory_allocation.c -o memory_allocation
chakrika@blue:~/OSSP/Week8$ ./memory_allocation
malloc() allocated 5 integers
calloc() allocated 5 integers and initialized them to zero
realloc() expanded arr1 to 10 integers
arr1 values: 1 2 3 4 5 6 7 8 9 10
Intentional memory leak created for Valgrind testing

```

Valgrind Analysis

```

chakrika@blue:~/OSSP/Week8$ valgrind --leak-check=full ./memory_allocation
==3678== Memcheck, a memory error detector
==3678== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3678== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3678== Command: ./memory_allocation
==3678==
malloc() allocated 5 integers
calloc() allocated 5 integers and initialized them to zero
realloc() expanded arr1 to 10 integers
arr1 values: 1 2 3 4 5 6 7 8 9 10
Intentional memory leak created for Valgrind testing
==3678==
==3678== HEAP SUMMARY:
==3678==   in use at exit: 400 bytes in 1 blocks
==3678==   total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated
==3678==
==3678== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==3678==   at 0x4b50858: malloc (vg_replace_malloc.c:447)
==3678==   by 0x40013CC: main (memory_allocation.c:59)
==3678==
==3678== LEAK SUMMARY:
==3678==   definitely lost: 400 bytes in 1 blocks
==3678==   indirectly lost: 0 bytes in 0 blocks
==3678==   possibly lost: 0 bytes in 0 blocks
==3678==   still reachable: 0 bytes in 0 blocks
==3678==   suppressed: 0 bytes in 0 blocks
==3678==
==3678== For lists of detected and suppressed errors, rerun with: -s
==3678== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Observation

The program demonstrates four dynamic memory management functions:

- **malloc()** — allocates a block of memory without initialization.
- **calloc()** — allocates memory and initializes it to zero.
- **realloc()** — changes the size of an already allocated memory block.
- **free()** — releases dynamically allocated memory.

Valgrind reports the intentionally unfreed leak pointer as memory that is **definitely lost**.

Result

The program was successfully implemented using malloc(), calloc(), realloc(), and free(). Valgrind was used to monitor heap memory and successfully identify the intentionally created memory leak.

QUESTION 2

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Aim

To demonstrate **Copy-on-Write (COW)** behavior after fork() and analyze memory usage before and after the child process modifies shared data.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define SIZE (4 * 1024 * 1024)

int data[SIZE / sizeof(int)];

void print_memory(const char *message) {
    FILE *fp;
    char line[256];

    printf("\n%s\n", message);

    fp = fopen("/proc/self/status", "r");

    if (fp == NULL) {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "VmRSS:", 6) == 0 ||
            strncmp(line, "VmSize:", 7) == 0) {
            printf("%s", line);
        }
    }

    fclose(fp);
}
```

Output

```
chakrika@blue:~/OSSP/Week8$ nano copy_on_write.c
chakrika@blue:~/OSSP/Week8$ gcc -o copy_on_write copy_on_write.c
chakrika@blue:~/OSSP/Week8$ ./copy_on_write

Parent PID: 3887

Parent memory before fork:
VmSize:      6864 kB
VmRSS:       6040 kB

Child PID: 3888

Child memory immediately after fork:
VmSize:      6864 kB
VmRSS:       5160 kB

Child modifying data...

Child memory after modifying data:
VmSize:      6864 kB
VmRSS:       5160 kB

Parent memory after child modification:
VmSize:      6864 kB
VmRSS:       6040 kB

Parent process completed.
```

Observation

Before `fork()`, the parent process has its own memory. After `fork()`, the parent and child initially share the same physical memory pages using Copy-on-Write.

When the child modifies the data array, the operating system creates private copies of the affected memory pages for the child. In this experiment, the `VmRSS` value does not show a visible increase after the modification. This is because `VmRSS` does not directly distinguish between shared and private memory pages.

The child modifies its own copy of the data, while the parent continues to use its original memory pages.

Memory Behavior

Stage	Parent	Child
Before fork()	Owns memory	Does not exist
Immediately after fork()	Shares pages	Shares pages
Child modifies data	Keeps original pages	Gets private copies
After modification	Original data remains	Modified copy is used

Result

The **Copy-on-Write mechanism after fork() was successfully demonstrated**. The child initially shares memory pages with the parent, and when the child modifies the data, private copies of the affected pages are created, demonstrating efficient memory management in Linux.